

ANDROID STUDIO

KOTLIN-KLASEAK

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Klase bat definitzen
2. Objektuak
3. Klase eta Objektuen arteko desberdintasuna
4. Propietateak
5. Funtzio gehitzea klaseari
6. Klase baten anatomia
7. Dena objektu bat da

KLASE BAT DEFINITZEN

Kotlinen, aldagaiak eta erlazionaturako funtzioak elkartu ditzazkegu **klaseak** erabiliz. Klase berri bat sortzeko adibide konkretu bat erabiliko dugu, Circle tipo berria sortuko dugu pausuz pausu.

Circle-k, aldagai eta funtzio hauek izango ditugu:

- ▶ **Radius** aldagaia
- ▶ **pi** Irakurketa hutseko aldagaia
- ▶ **Circumference()** funtzioa

Guztia batera sortu beharean , klase hau poliki-poliki sortuko dugu.

KLASE BAT DEFINITZEN

Klase huts bat

Lehenik, definitu dezagun klase huts bat, aldagairik eta funtziorik gabe.

```
class Circle
```

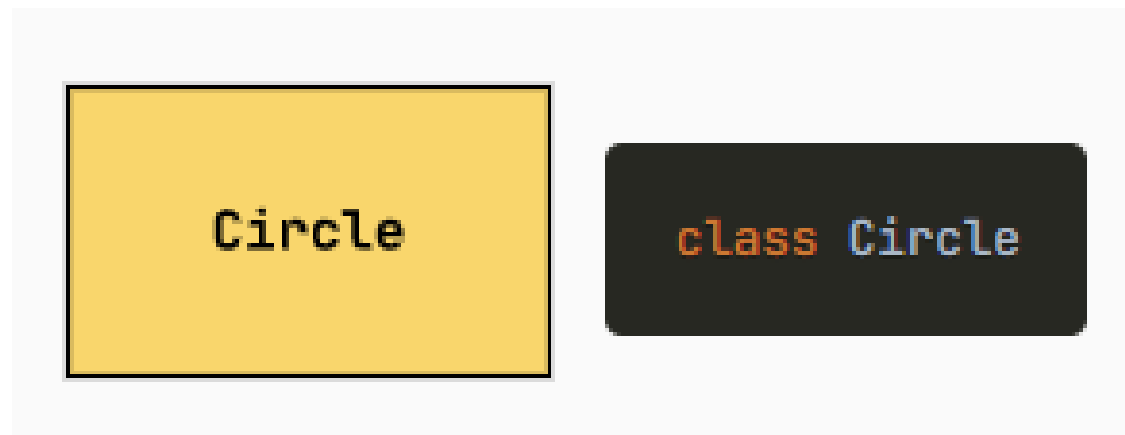
Hori da Circle izeneko klase berri bat sortzeko behar den guztia. Klase bat sortzen dugunean, aldagai mota(tipo) berri bat sortzen ari gara, Int, Double eta String bezalakoak. Beste era batera esanda, behin Circle klasea horrela definituta, mota bat normalean kokatzen dugun edozein lekutan erabil dezakegu, funtzio-parametro gisa esaterako.

```
fun draw(circle: Circle) {  
    // Code that draws the circle would go here  
}
```

KLASE BAT DEFINITZEN

Askotan, baliagarria izaten da klaseen diagramak egitea, horiek bistaratu ahal izateko eta ideiak gelako kideei komunikatzeko. Diagramak egiteko, diagramazio-estandarra erabiliko dugu: modelatze-lengoaia bateratua edo **UML**.

Erraz hasiko gara. Klase bat erakusteko, klasearen izena koadro batean jarriko dugu:



KLASE BAT DEFINITZEN

Gure Circle mota berria daukagunez, aldagai bat sor genezake gordetzeko, baina gure klasea erabat hutsik dagoenez, oraindik ez da bereziki erabilgarria. Beraz, hori egin aurretik, propietate bat emango diogu Circle-ri, radius.

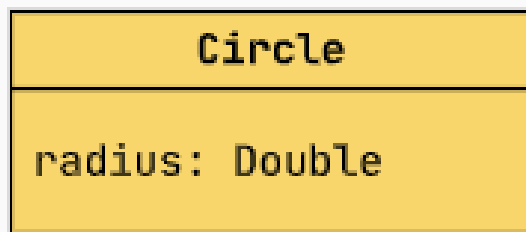
Bizitza errealean, zirkulu bakoitzak erradio bat duenez, gure kodeko Circle elementu bakoitzak radius aldagai bat ere baduela ziurtatu behar dugu.

```
class Circle(var radius: Double)
```

KLASE BAT DEFINITZEN

Hemen, klaseari aldagai berri bat gehitzen diogu, radius izenekoa, Double motakoa. Radius-en balioa aldatu egin daiteke, lehenago datorren var-a dela eta. Kotlinen, honelako klase batean aldagaia **klasearen propietatea** deitzen da.

Gure diagrama ere eguneratko dugu:



```
class Circle(var radius: Double)
```

Orain erradio bat duen zirkulu mota bat daukagunez, prest gaude erabiltzen hasteko!

OBJEKTUAK

Hasteko, double aldagai bati mota bat adierazi eta esleituko diogu:

```
val radiusOfSmallCircle: Double = 5.2
```

Klase bat sortu dugunean, Circle izeneko tipo berri bat sortu dugu. Double motako aldagai bat izan dezakezun bezala, Circle motako aldagai bat ere izan dezakezu. Erraza da Circle aldagai bat adieraztea eta esleitzea:

```
val smallCircle = Circle(5.2)
```


OBJEKTUAK

Horrek smallCircle izeneko aldagai berri bat sortzen du, eta 5.2ko erradioa duen Circle bat esleitzen zaio. Kontuan izan Double balio asko izan ditzakeela, hala nola, 5.2, 6.7, 10.0. Era berean, Circle balio asko izan ditzakezu...

5.2ko erradioa duen zirkulu bat

6.7ko erradioa duen zirkulu bat

10.0ko erradioa duen zirkulu bat

Baina klaseak direnean, balioak deitu beharrean, normalean **objektuak** deitzen diegu.

OBJEKTUAK

Objektuak eraikitzen

Laburbilduz, orainarte:

```
// Declaring the class
class Circle(var radius: Double)

// Using the class
val smallCircle = Circle(5.2)
```

Zer gertatzen da **Circle (5.2)** idazten dugunean?

Circle() izeneko funtzio bati deitzen ari bagina bezala da, radius izeneko parametro bat eta Circle itzulera-mota bat duena. Hala ere, funtzio mota horri ez zaio funtzio deitzen. **Eraikitzaile** deitzen zaie objektu berri bat eraikitzen dutelako.

OBJEKTUAK

Eraikitzailearen parametroak

Kontuan izan eraikitzaile bati deitzen dionean, eraikitzailearen irekiera- eta itxiera-parentesien artean dagoen propietate bakoitzerako argumentu bat eman behar zaiola, **var radius**: Double jartzen dugunez parentesi horien artean, Double motako argumentu bat eman behar diogu eraikitzaileari deitzen diogunean:

```
class Circle(var radius: Double)
val smallCircle = Circle(5.2)
```



The diagram illustrates the relationship between the class constructor parameter and the object creation call. A dashed line connects the `radius` parameter in the `Circle` class constructor to the `5.2` argument in the `Circle(5.2)` call. A solid white arrow points from the `5.2` argument to the `radius` parameter, indicating the value being passed.

OBJEKTUAK

Kontuan izan radius bi funtzio betetzen ari dela:

- ▶ Eraikitzailearen parametro bat da. Balio bat eman behar diozu eraikitzaileari deitzen diozun bakoitzean.
- ▶ Propietate bat da.

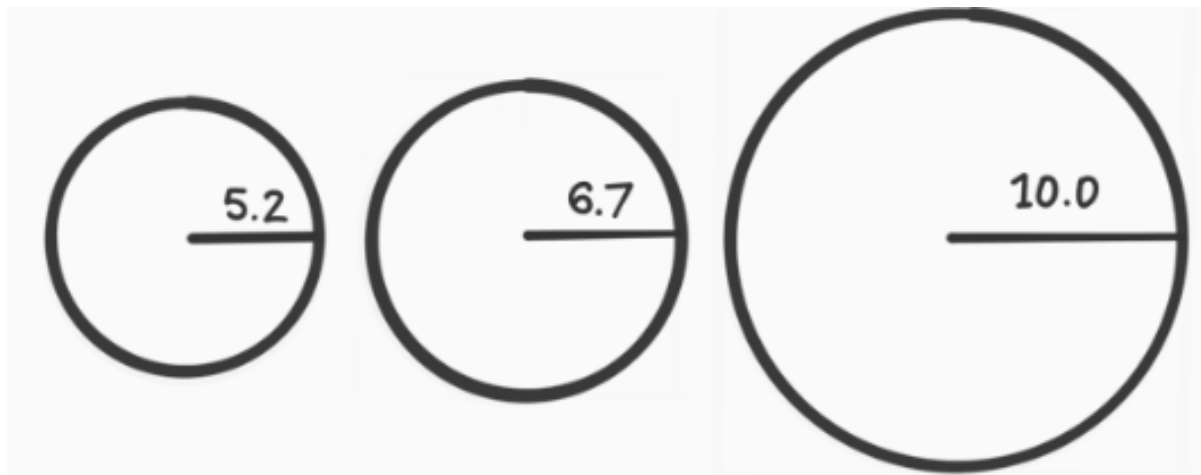
Objektu bat sortzen duzunean, **klasearen instantzia** bat sortzen ari zara.

Horregatik, objektu bat sortzeari **klasearen instantziazioa** esaten zaio batzuetan.

KLASE ETA OBJETUEN ARTEKO DESBERDINTASUNA

Klase batek kontzeptu baten ezaugarriak eta portaera deskribatzen ditu. Zirkuluez ari bagara, ezaugarri horiek erradioa, diametroa, zirkunferentzia eta azalera izan daitezke.

Objektu bat gauza horren instantzia errealak da. Hemen hiru objektu zirkular daude:



KLASE ETA OBJETUEN ARTEKO DESBERDINTASUNA

Zirkulu klase batek honelako galderak erantzuten ditu:

- ▶ Zer esan nahi du zerbait zirkulu bat izatea?
- ▶ Zer ezaugarri ditu?
- ▶ Zeregiten du?

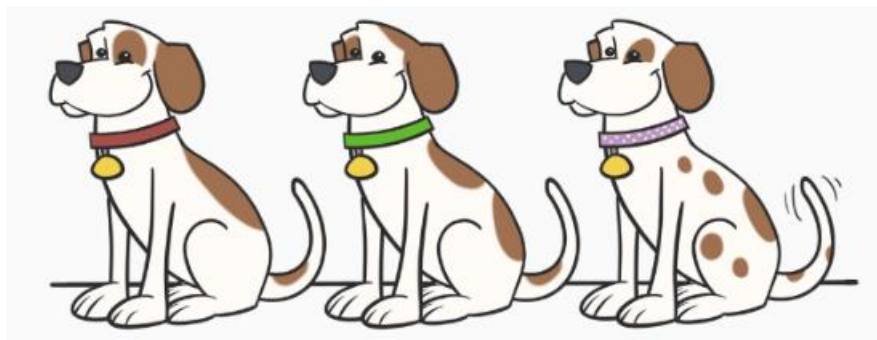
Objektu zirkular batek honako galdera hauei erantzuten die:

- ▶ Zein da zirkulu honen erradioa?
- ▶ Zein da zirkunferentzia?
- ▶ Zein da eremua?

KLASE ETA OBJETUEN ARTEKO DESBERDINTASUNA

Klaseak eta objektuak bereizten laguntzeko adibide gehiago:

- ▶ Baliteke **zenbaki** mota bat izatea, **32.768** edo **6.62607015** bezalako objektuekin.
- ▶ Baliteke **Kolore** klasea izatea, **gorria**, **berdea** eta **urdina** bezalako objektuekin.
- ▶ **Txakur** mota bat izan dezakezu, **Fido**, **Rover** edo **Wagglytails** bezalako objektuekin.



PROPIETATEAK

Propietate baten balioa lortzea

Objektu zirkular bat sortu dugu, nola lor dezakegu haren erradioa?
Erraza: objektu baten propietate baten balioa lortzeko, idatzi aldagaiaren izena, puntu bat eta propietatearen izena, honela:

```
val smallCircle = Circle(5.2)  
val radiusOfSmallCircle: Double = smallCircle.radius
```

Kode hori exekutatu ondoren, radiusOfSmallCircle 5.2 izango da.

PROPIETATEAK

Propietate konstanteak

Pi jar genezake parentesien barruan radius-ekin egin genuen bezala, koma batekin bereiziz:

```
class Circle(var radius: Double, val pi: Double)
```

Baina hau egiten badugu, orduan pi eman beharko genuke zirkulu baten instantzia bat sortzen dugun bakoitzean, honela:

```
val smallCircle = Circle(5.2, 3.14)  
val mediumCircle = Circle(6.7, 3.14)  
val largeCircle = Circle(10.0, 3.14)
```

PROPIETATEAK

Hori ez da guk nahi duguna. Pi beti balio bera izan beharko luke zirkulua edozein dela ere, ez du zentzurik eraikitzailearen parametro bat izatea. Izan ere, hobe litzateke dei-kodeak ezin badu inoiz pi balioa zehaztu Circle bat eraikitzean. Horretarako, parentesi barrutik pi atera behar dugu:

```
class Circle(var radius: Double) {  
    val pi: Double = 3.14  
}
```

PROPIETATEAK

Parentesietatik ateratzean, pi propietatea jada ez da eraikitzailearen parametro bat, eta, beraz, eraikitzaileari berdin deitzen jarraituko dugu:

```
val smallCircle = Circle(5.2)
```

Propietate pribatuak

Orain dagoen bezala, edozein zirkuluko bai radius zein pi balioa lor ditzazkegu:

```
val smallCircle = Circle(5.2)

val radiusOfSmallCircle = smallCircle.radius
val piFromSmallCircle = smallCircle.pi
```

PROPIETATEAK

Ez zait arrazoi askorik bururatzen kasetik kanpoko kodeak pi balioa beharko lukeenik. Egin dezagun propietate hau klasearen barrutik bakarrik ikus dadin. Horretarako, `private` hitza gehi dezakegu aldagaia deklaratzeko dugunean.

```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
}
```

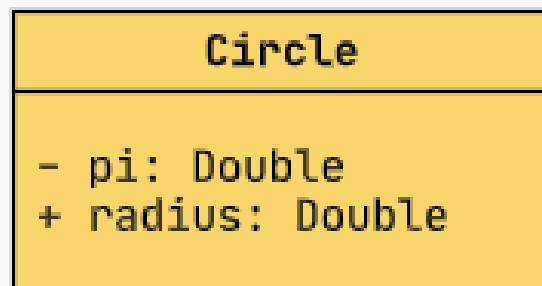
Orain, pi propietatearen balioa klasearen gorputzaren kanpotik lortzen saiatzen bagara, errorea emango digu:

```
val smallCircle = Circle(5.2)  
  
val radiusOfSmallCircle = smallCircle.radius  
val piFromSmallCircle = smallCircle.pi
```

PROPIETATEAK

Eguneratu dezagun gure UML klase-diagrama pi propietatea sartzeko. Propietate baten ikuspena diagraman adieraz dezakegu:

- ▶ Propietate pribatuen aurretik – sinboloa erabiltzen da
- ▶ Propietate publikoen aurretik + sinbolo erabiltzen da



```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
}
```

FUNTZIO GEHITZEA KLASEARI

Funtzio bat klase batekoa denean, sarritan **metodo** edo **funtzio kide** deitzen zaio. Erraza da metodo bat gehitzea klase bati: klaseko gorputzean jartzea besterik ez da egin behar:

```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
  
    fun circumference(radius: Double) = 2 * pi * radius  
}
```

FUNTZIO GEHITZEA KLASEARI

Lehen aldiz circumference () funtzioa sortu genuenean, radius izeneko parametro bat izateak zentzua zuen. Baina funtzio hori klaseari gehitzen ari garenez, radius propietatearen balioari erreferentzia egin diezaiokegu.

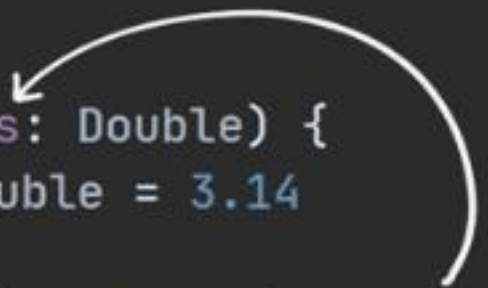
```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
    fun circumference(radius: Double) = 2 * pi * radius  
}
```



FUNTZIO GEHITZEA KLASEARI

Funtzioaren radius parametroa ken dezakegu, propietateari erreferentzia egin diezaion.

```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
  
    fun circumference() = 2 * pi * radius  
}
```



FUNTZIO GEHITZEA KLASEARI

Objektu batean funtzio bati deitzea radius balioa lortu genuenaren antzera egiten da: aldagaiaren izena, puntu bat eta funtzioaren izena.

```
. val smallCircle = Circle(5.2)  
  val circumferenceOfSmallCircle: Double = smallCircle.circumference()
```

FUNTZIO GEHITZEA KLASEARI

Eta jarraitu aurretik, gehitu diezaion circumferance() funtzioa gure diagramari!

Circle
- pi: Double + radius: Double
+ circumference(): Double

```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
  
    fun circumference() = 2 * pi * radius  
}
```

FUNTZIO GEHITZEA KLASERARI

Gehitu funtzio gehiago

```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
  
    fun circumference() = 2 * pi * radius  
    fun area() = pi * radius * radius  
    fun diameter() = 2 * radius  
}  
  
val smallCircle = Circle(5.2)  
val diameterOfSmallCircle = smallCircle.diameter()
```

FUNTZIO GEHITZEA KLASEARI

```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
  
    fun circumference() = diameter() * pi  
    fun area() = pi * radius * radius  
    fun diameter() = 2 * radius  
}
```

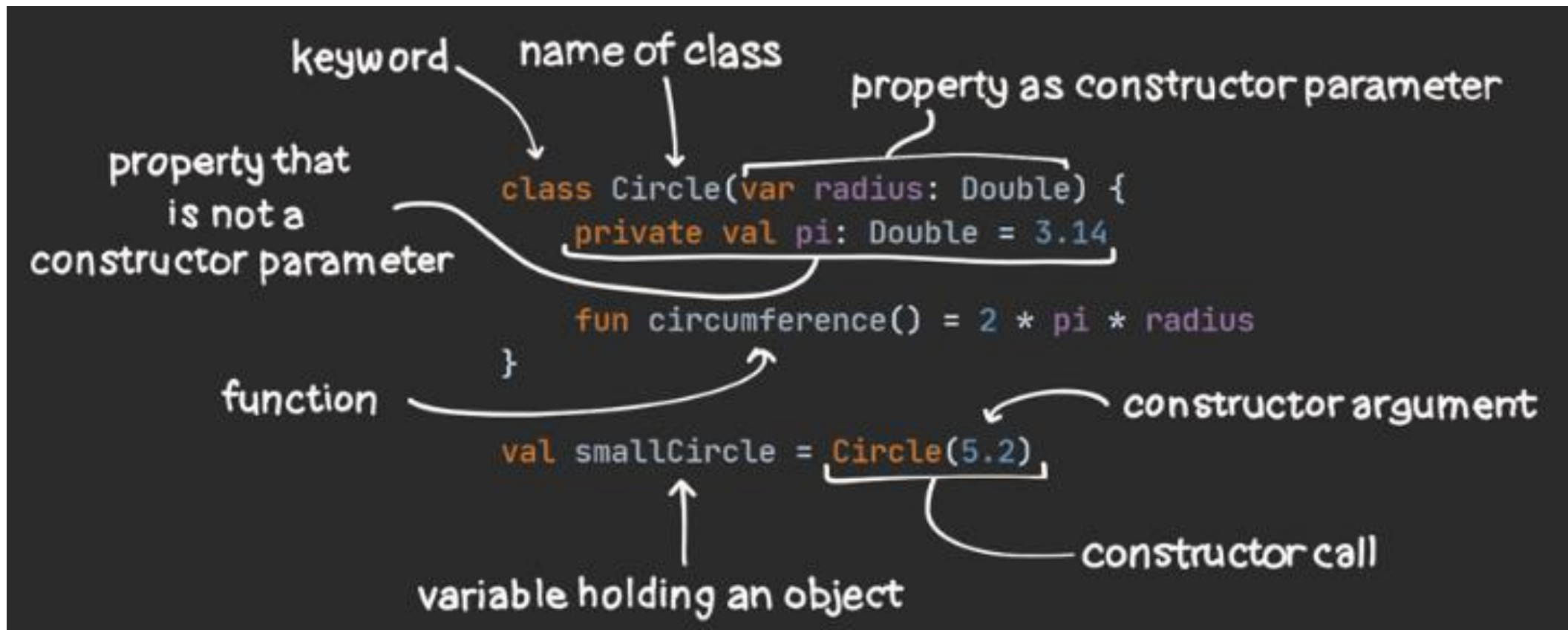
FUNTZIO GEHITZEA KLASERARI

UML

Circle
- pi: Double + radius: Double
+ circumference(): Double + area(): Double + diameter(): Double

```
class Circle(var radius: Double) {  
    private val pi: Double = 3.14  
  
    fun circumference() = diameter() * pi  
    fun area() = pi * radius * radius  
    fun diameter() = 2 * radius  
}
```

KLASE BATEN ANATOMIA



DENA OBJEKTU BAT DA

Orain arte Kotlin-eko mota integratuak baino ez ditugu erabili, hala nola Double, String edo Boolean. Baina badakizue mota horiek erabiltzen ditugunean klaseak eta objektuak erabiltzen ditugula. Gure Circle objektuaren propietateak lortzeko eta funtzioak deitzeko puntua erabiltzen dugun bezala, puntu bat ere erabil dezakegu mota horietako edozeinetan.

Double objektu bezala

```
fun circumference() = 2 * pi * radius
```

```
fun circumference() = 2.times(pi).times(radius)
```

DENA OBJEKTU BAT DA

Kateak objetu bezala

```
val greeting: String = "Welcome"

val numberOfLettersInGreeting = greeting.length // Evaluates to 7
val loudGreeting = greeting.toUpperCase()        // Evaluates to "WELCOME"
val substring = greeting.drop(3)                 // Evaluates to just "come"
```


DENA OBJEKTU BAT DA

Boolearrak objetu bezala

```
val isDark: Boolean = true
val isRaining: Boolean = false

val shouldTurnOnHeadlights = isDark.or(isRaining)
val shouldStayHome = isDark.and(isRaining)
```

DENA OBJEKTU BAT DA

Praktika onak

Aldagai boolear batean `or()` eta `and()` funtzioak erabil daitezkeen arren, normalean hobe da `||` eta `&&` operadoreak erabiltzea horien ordeez, honela:

```
val shouldTurnOnHeadlights = isDark || isRaining // Evaluates to true
val shouldStayHome = isDark && isRaining           // Evaluates to false
```